TextToAutoML

System architecture | Repository-grounded component and deployment view

This document describes the implemented frontend, FastAPI services, model flows, filesystem persistence, and production routing in this repository. It distinguishes the tabular AutoML pipeline from the separate image-classification pipeline.

Request and Routing Architecture

React frontend
`frontend/src/App.jsx`
Uploads datasets, gathers task intent, displays results, and sends prediction requests using same-origin paths.

→

Nginx production frontend
`Dockerfile.frontend`, `nginx.conf`
Serves the Vite production bundle and reverse-proxies API paths to the backend service.

→

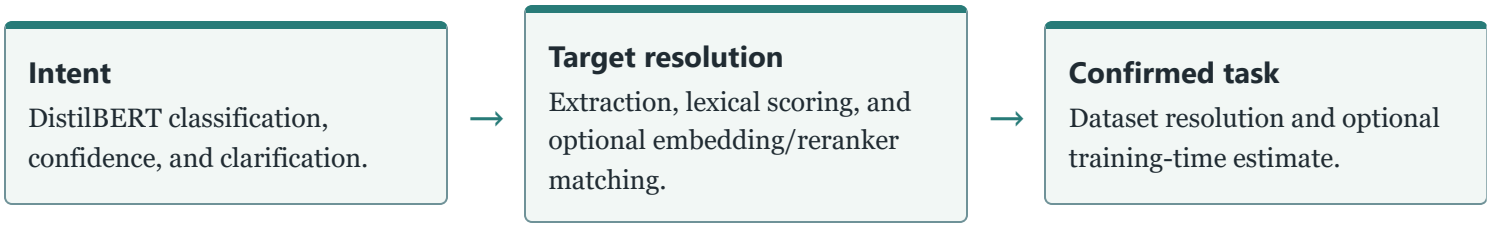
FastAPI application
`backend/main.py`
Registers API routers, CORS, liveness, and model-aware readiness.

Route group	Implemented responsibility	Primary code
`/upload/`	Accepts tabular datasets and stores uploaded files.	`backend/api/upload.py`
`/nlp/analyze`, `/nlp/train`, `/nlp/diagnose-target-proxies`	Resolves a natural-language task and target; optionally trains or diagnoses target leakage.	`backend/api/nlp.py`
`/nlp/route`	Routes requests between supported workflow intents, including image workflows.	`backend/api/nlp_routing.py`
`/train/`, `/predict/`	Direct tabular training and persisted-model prediction.	`backend/api/train.py`, `backend/api/predict.py`
`/image/analyze`, `/image/train`, `/image/predict`	Image ZIP inspection, image-classifier training, and single-image prediction.	`backend/api/image_classification.py`
`/health`, `/ready`	Lightweight process/storage liveness and required intent-artifact readiness.	`backend/main.py`

Natural-Language and Model Flow

> **Intent classifier:** DistilBERT tokenizer and sequence-classification weights are loaded from the single configured `INTENT_MODEL_DIR`. Its label mapping is stored beside the weights. Readiness checks artifact structure without initializing model weights; request-time loading reports a typed unavailable error.

The NLP pipeline runs intent detection, confidence/clarification analysis, explicit task mapping, target extraction, and dataset resolution. Strong lexical target matches can resolve without semantic models. When semantic comparison is needed, target matching uses a local embedding model and a cross-encoder reranker with local-only loading.



Model names are configurable as `TARGET_EMBEDDING_MODEL` and `TARGET_RERANKER_MODEL`. These semantic dependencies are optional for application readiness, but an NLP request that requires unavailable semantic matching returns HTTP 503.

Tabular AutoML

The direct training route and NLP integration both call the existing tabular AutoML orchestration. CPU-bound training work is dispatched from async API handlers with worker threads so liveness requests remain serviceable.

Dataset preparation
Load, validate, select features, and prepare target/group columns.

→

Train and evaluate
Split train/validation/test data, preprocess, train candidates, tune where requested, and compare validation metrics.

→

Select and persist
Choose the best candidate, refit as implemented, report holdout metrics, and save the model package.

Candidate orchestration and evaluation live under `backend/services/automl`. The selected estimator and reconstruction metadata are persisted as a Joblib package under `storage/models`; prediction reloads that package through the persistence service. NLP training can additionally record run comparisons in `storage/reports/training_history.json`.

Image Classification

Image classification is a separate PyTorch workflow, not part of tabular AutoML. The API accepts a ZIP dataset, extracts and analyzes its class folders and any predefined splits, then trains the implemented MobileNetV2 classifier on train/validation/test partitions.

Image dataset
ZIP extraction, class mapping, split analysis, and image validation.

→

MobileNetV2
Image transforms, training, validation, and held-out test evaluation.

→

Image package
PyTorch state dictionary plus JSON manifest with architecture, classes, preprocessing, and metrics.

Image training and prediction code is under `backend/services/image_classification`. Packages are stored under `storage/image_models`; the manifest is used to reconstruct the classifier and class ordering for prediction.

Storage and Production Deployment

Path / setting	Contents or role
`storage/uploads`	Uploaded tabular datasets.
`storage/models`	Joblib tabular model packages.
`storage/image_models`	Image model weights and manifests.
`INTENT_MODEL_DIR`	Trained DistilBERT inference artifacts; shared by training, serving, and readiness.
`HF_CACHE_DIR`	Host-mounted Hugging Face cache used for local semantic target-matching models.
`storage/reports/training_history.json`	Training run history and comparison metadata.

Compose builds a Python backend image and a separate multi-stage frontend image. The frontend image builds the Vite bundle and serves it through Nginx; Nginx forwards same-origin API paths to the Compose backend service. The backend image installs application requirements from PyPI and also configures the PyTorch CPU wheel index for CPU builds.

`/health` checks service liveness without loading models. `/ready` requires uploads storage and structurally complete intent-classifier artifacts; semantic matching models are reported as optional and are loaded only when needed.

Source map: `backend/api`, `backend/services`, `frontend/src`, `Dockerfile`, `Dockerfile.frontend`, `docker-compose.yml`, and `nginx.conf`.